

1. What is Dimensionality Reduction?

Definition: Reducing the number of features in a dataset while retaining essential information.

Purpose: Simplifies models, enhances performance, improves visualization, and reduces computational costs.

2. The Curse of Dimensionality

Definition: High-dimensional data can lead to sparse data and overfitting, making it hard for models to generalize.

Impact: Increased model complexity and difficulty in pattern recognition.

3. Types of Dimensionality Reduction

Feature Selection: Choosing the most relevant features (e.g., using filter, wrapper, or embedded methods).

Feature Extraction: Transforming data into a lower-dimensional space (e.g., PCA, t-SNE, LDA, Autoencoders).

4. Common Techniques

- PCA (Principal Component Analysis):

Linear: Projects data onto directions of maximum variance.

Use: Ideal for linear separability and variance retention.

- t-SNE:

Non-linear: Preserves local structure for visualization. Use: Best for visualizing complex data in 2D or 3D.

- LDA (Linear Discriminant Analysis):

Supervised: Maximizes class separability. Use: Effective for classification tasks.

- Autoencoders:

Neural Networks: Learn compressed, non-linear representations. Use: Useful for anomaly detection and unsupervised learning.

5. Challenges

Information Loss: Some data loss is inevitable.

Interpretability: Reduced features may lack clear meaning.

Overfitting: Must be balanced to avoid losing useful information.

Overfitting:

Problem: The model is too complex and learns noise in the training data.

Result: High accuracy on training data, poor generalization to new data.

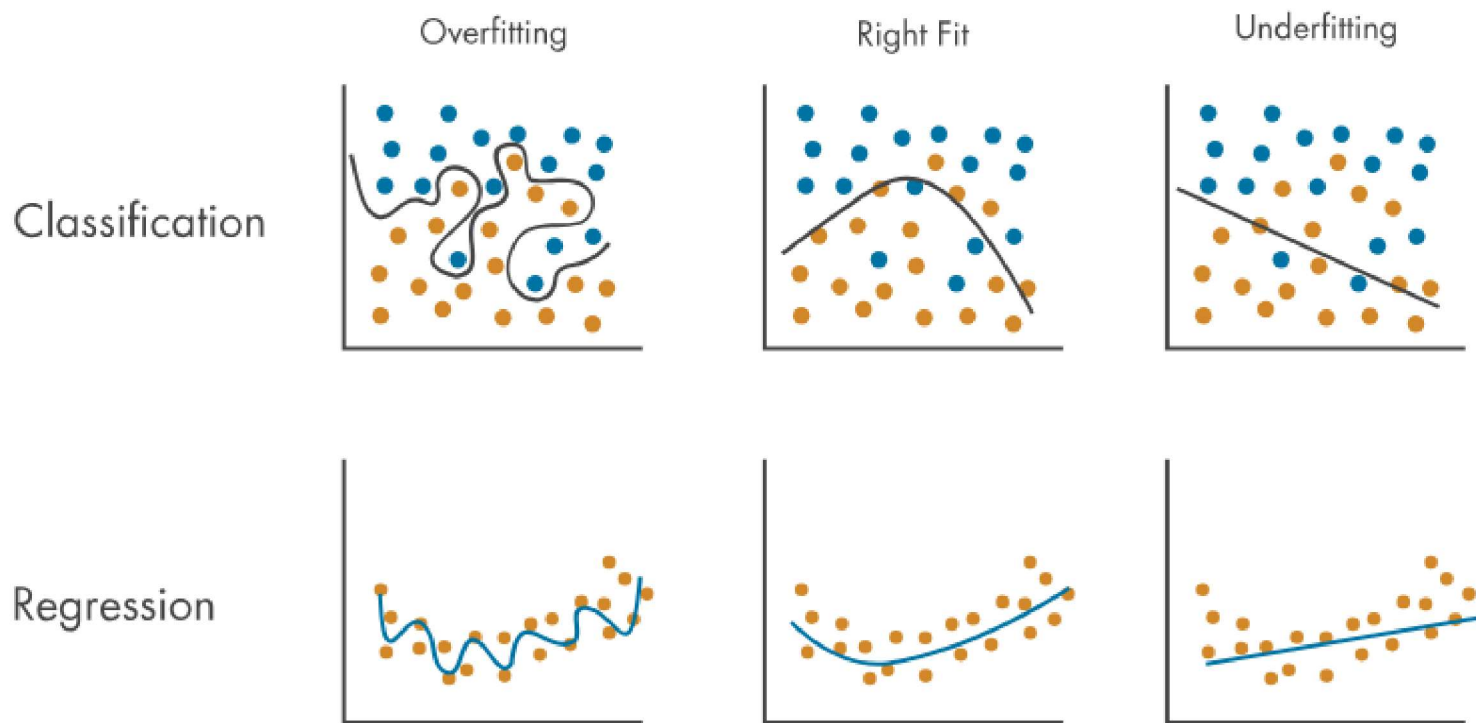
Solution: Simplify the model, use regularization, or gather more data.

Underfitting:

Problem: The model is too simple to capture the underlying patterns in the data.

Result: Poor accuracy on both training and test data.

Solution: Increase model complexity, add more features, or reduce regularization.



```
In [13]: import seaborn as sns
import pandas as pd

# Load the Iris dataset
df = sns.load_dataset("iris")

df
```

```
Out[13]:
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

150 rows × 5 columns

```
In [14]: feat=df.iloc[:, :-1]
feat
```

```
Out[14]:
```

	sepal_length	sepal_width	petal_length	petal_width
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
...	...	...	...	...
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

150 rows × 4 columns

```
In [43]: from sklearn.preprocessing import StandardScaler
```

```
# Assuming X is your feature matrix  
scaler = StandardScaler()  
X = scaler.fit_transform(feats)  
X
```

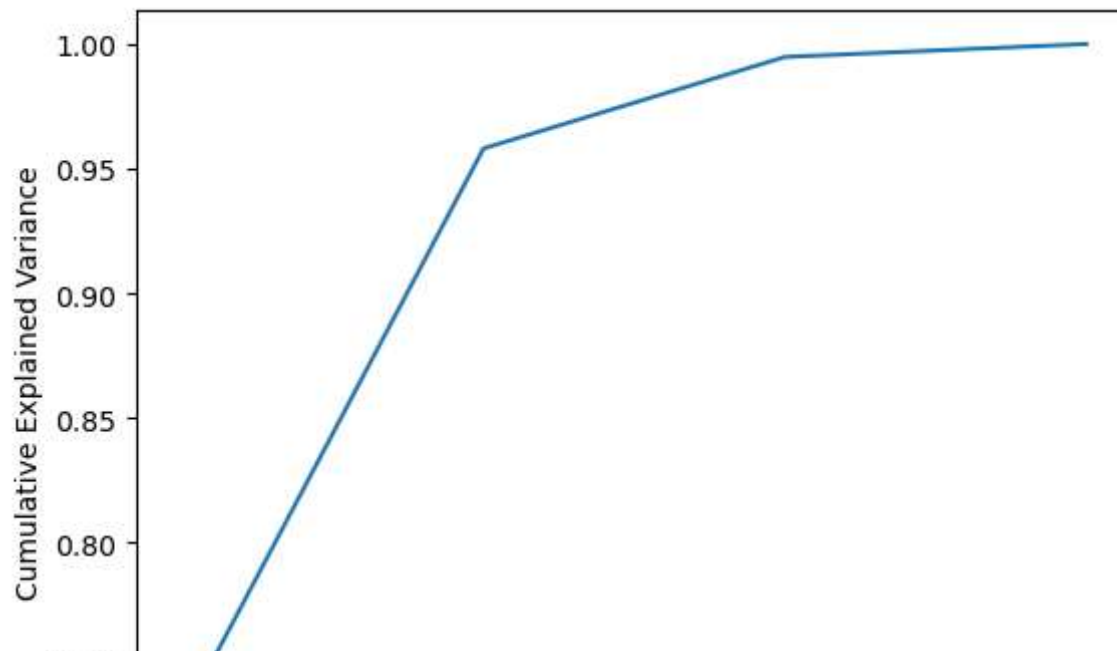
```
Out[43]: array([[ -9.00681170e-01,  1.01900435e+00, -1.34022653e+00,  
                -1.31544430e+00],  
               [-1.14301691e+00, -1.31979479e-01, -1.34022653e+00,  
                -1.31544430e+00],  
               [-1.38535265e+00,  3.28414053e-01, -1.39706395e+00,  
                -1.31544430e+00],  
               [-1.50652052e+00,  9.82172869e-02, -1.28338910e+00,  
                -1.31544430e+00],  
               [-1.02184904e+00,  1.24920112e+00, -1.34022653e+00,  
                -1.31544430e+00],  
               [-5.37177559e-01,  1.93979142e+00, -1.16971425e+00,  
                -1.05217993e+00],  
               [-1.50652052e+00,  7.88807586e-01, -1.34022653e+00,  
                -1.18381211e+00],  
               [-1.02184904e+00,  7.88807586e-01, -1.28338910e+00,  
                -1.31544430e+00],  
               [-1.74885626e+00, -3.62176246e-01, -1.34022653e+00,  
                -1.31544430e+00],  
               [-1.14301691e+00,  9.82172869e-02, -1.28338910e+00,
```

```
In [16]: # PCA
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

pca = PCA()
pca.fit(X)
import numpy as np
# Explained variance
explained_variance = pca.explained_variance_ratio_

# Cumulative explained variance
cumulative_explained_variance = np.cumsum(explained_variance)

# Plot
plt.plot(range(1, len(cumulative_explained_variance) + 1), cumulative_explained_variance)
plt.xlabel('Number of Components')
plt.ylabel('Cumulative Explained Variance')
plt.show()
```



```
In [29]: range(1, len(cumulative_explained_variance) +1)
```

```
Out[29]: range(1, 5)
```

```
In [28]: for i in range(1,5):  
         print(i)
```

```
1  
2  
3  
4
```

```
In [50]: import seaborn as sns
import pandas as pd

# Initialize PCA
pca = PCA(n_components=2) # Reduce to 2 dimensions

# Fit and transform the data
pca_data = pca.fit_transform(X)

# Convert to a DataFrame for easier plotting
pca_df = pd.DataFrame(pca_data, columns=['PC1', 'PC2'])
pca_df['species'] = df['species']

pca_df
```

```
Out[50]:
```

	PC1	PC2	species
0	-2.264703	0.480027	setosa
1	-2.080961	-0.674134	setosa
2	-2.364229	-0.341908	setosa
3	-2.299384	-0.597395	setosa
4	-2.389842	0.646835	setosa
...	...	...	...
145	1.870503	0.386966	virginica
146	1.564580	-0.896687	virginica
147	1.521170	0.269069	virginica
148	1.372788	1.011254	virginica
149	0.960656	-0.024332	virginica

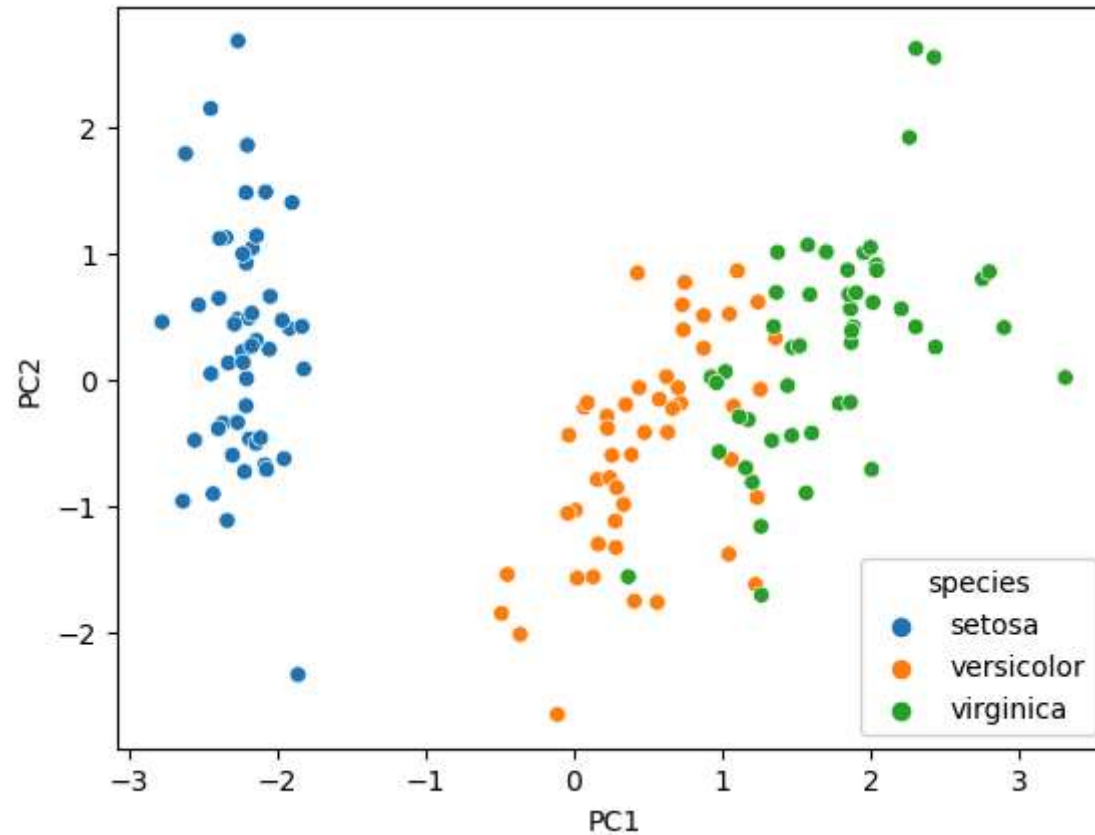
150 rows × 3 columns

```
In [51]: # Plot the reduced data
sns.scatterplot(pca_df["PC1"],pca_df["PC2"],hue=pca_df["species"])
```

C:\Users\HP\anaconda3\lib\site-packages\seaborn_decorators.py:36: FutureWarning: Pass the following variables as keyword args: x, y. From version 0.12, the only valid positional argument will be `data`, and passing other arguments without an explicit keyword will result in an error or misinterpretation.

```
warnings.warn(
```

```
Out[51]: <AxesSubplot:xlabel='PC1', ylabel='PC2'>
```



```
In [33]: pca_df
```

```
Out[33]:
```

	PC1	PC2	species
0	-2.264703	0.480027	setosa
1	-2.080961	-0.674134	setosa
2	-2.364229	-0.341908	setosa
3	-2.299384	-0.597395	setosa
4	-2.389842	0.646835	setosa
...	...	...	...
145	1.870503	0.386966	virginica
146	1.564580	-0.896687	virginica
147	1.521170	0.269069	virginica
148	1.372788	1.011254	virginica
149	0.960656	-0.024332	virginica

150 rows × 3 columns

```
In [34]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
In [38]: # Feature matrix
X = pca_df[['PC1', 'PC2']]

# Target variable (species)
y = pca_df['species']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
In [39]: # Initialize the Logistic Regression model
model = LogisticRegression()

# Fit the model on the training data
model.fit(X_train, y_train)
```

```
Out[39]: LogisticRegression()
```

```
In [40]: # Predicting the target values for the test set
y_pred = model.predict(X_test)
```

```
In [41]: # Accuracy score
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy:.2f}')

# Classification report
print('Classification Report:')
print(classification_report(y_test, y_pred))

# Confusion matrix
print('Confusion Matrix:')
print(confusion_matrix(y_test, y_pred))
```

Accuracy: 0.90

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.88	0.78	0.82	9
virginica	0.83	0.91	0.87	11
accuracy			0.90	30
macro avg	0.90	0.90	0.90	30
weighted avg	0.90	0.90	0.90	30

Confusion Matrix:

```
[[10  0  0]
 [ 0  7  2]
 [ 0  1 10]]
```

Here are the main points of feature engineering:

1. Feature Selection:

Identify and select the most important features using methods like filtering, wrapping, or embedded techniques.

2. Feature Creation:

Generate new features from existing data, such as polynomial features, interaction terms, or datetime features.

3. Feature Transformation:

Modify features to improve their suitability for modeling, including normalization, standardization, and log transformation.

4. Handling Missing Values:

Address missing data using imputation or removal to prevent negative impacts on model performance.

5. Handling Outliers:

Manage extreme values using methods like capping, transformation, or outlier removal.

6. Dimensionality Reduction:

Reduce the number of features while retaining important information using techniques like PCA or t-SNE.

In []: